PROJECT DOCUMENTATION

SPENDWISE

1.INDRODUCTION

- Project Title - Spendwise
- Team Leader - SWTID-2026-4342
- TERM LEADER : ARTHI.R
- TERM MEMBERS: GAYATHRI.V.S
- Sruthi.U
- Varshitha.N
- Yamuna.P

2. Project Title

SpendWise – A Simple Backend Learning Project using Node.js, Express, MongoDB, and JWT

3. Project Overview

SpendWise is a backend application designed to help beginner developers learn how to build secure REST APIs using Node.js, Express, MongoDB, and JWT authentication. The project focuses on implementing user authentication and basic CRUD operations in a clean and simple way so students can understand how backend systems work in real-world scenarios.

4. Key Objectives:

- Provide secure user authentication using JWT
- Enable basic CRUD operations on user data (transactions/records)
- Demonstrate MongoDB integration using Mongoose
- Show how protected routes work using middleware
- Maintain a simple and beginner-friendly backend architecture

5. Target Users:

- Beginner backend developers
- Students learning Node.js and Express
- Freshers preparing for technical interviews
- Anyone who wants a simple project to understand JWT and MongoDB
- Learners who want hands-on practice with REST APIs

6. Project Instruction

SpendWise is a backend-only project built using Node.js and Express.

7. Quick Start:

npm init

npm install

npm run dev

8. **Steps to Create the Project:**

- Choose or create a directory for the backend project.
- Open terminal and navigate to that directory.
- Initialize Node.js project using npm init.
- Install required packages: express, mongoose, jsonwebtoken, bcrypt, cors, dotenv.
- Create folders: models, routes, controllers, middleware, config.
- Start the server using npm run dev.
- Test APIs using Postman or Thunder Client at http://localhost:5000.

9. Pre-requisites:

Software Requirements

Software	Purpose
Node.js	JavaScript runtime for backend
MongoDB	NoSQL database for data storage
npm	Package manager
Git	Version control system
Code Editor	VS Code or similar

10. Knowledge Requirements

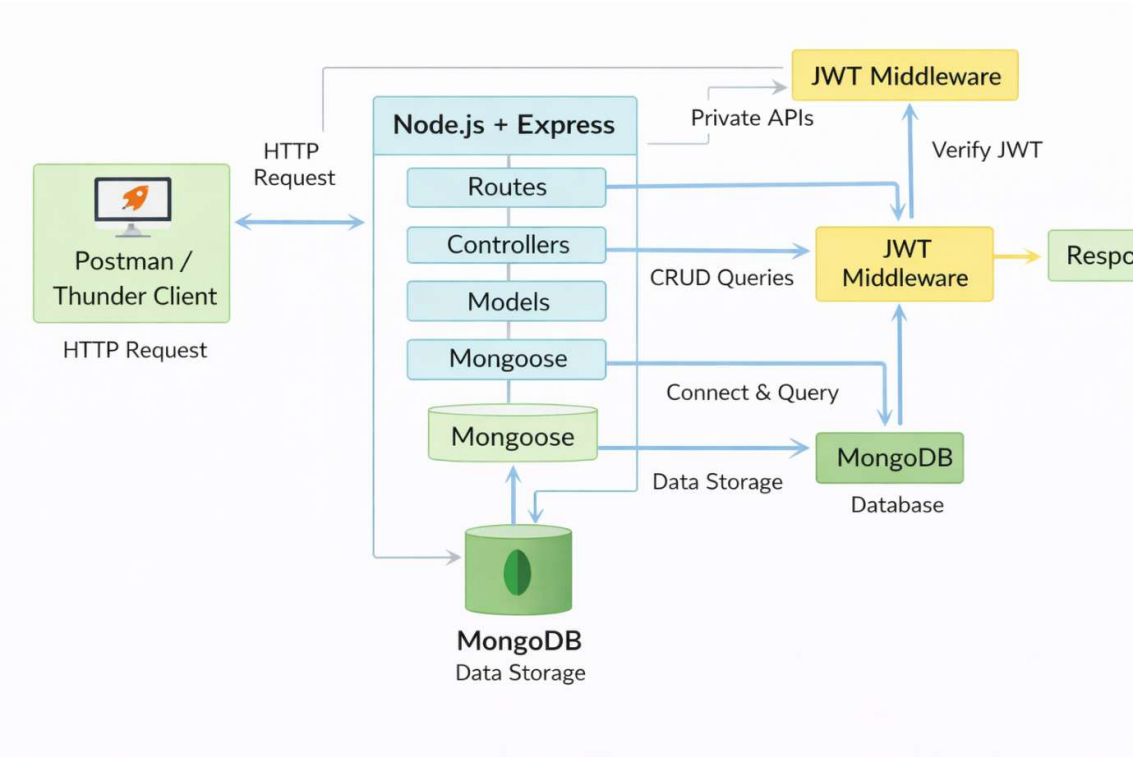
- JavaScript ES6+
- Node.js and Express.js
- MongoDB and Mongoose
- JWT authentication
- Basic REST API concepts
- Git basics

11. Thumbnail for Project

- A simple backend project thumbnail representing:

Node.js + Express + MongoDB + JWT + CRUD APIs

12. Project Architecture Image



13. Technical Architecture

Backend Architecture (Node.js + Express)

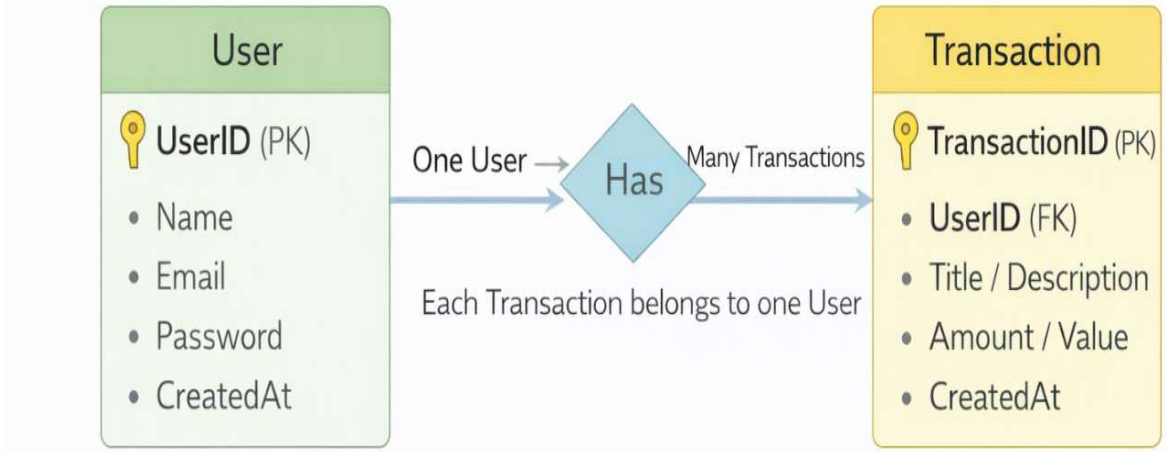
- **Controllers:** Handle business logic for authentication and CRUD
- **Models:** Mongoose schemas for User and Transaction

- **Routes:** API endpoints for auth and CRUD operations
- **Middleware:** JWT verification and request protection
- **Configuration:** Database connection and environment variables

14. Database Architecture (MongoDB)

- Users Collection:
  _id, name, email, password (hashed), createdAt
- Transactions Collection:
  _id, userId (FK), title/description, amount/value, createdAt
- Relationship:
  Many Transactions belong to One User

15. ER Diagram



- One User ⟶ Many Transactions
- Each Transaction belongs to one User

16. Features

#	Feature	Description
1	User Authentication	JWT-based registration and login
2	CRUD Operations	Create, Read, Update, Delete transactions
3	Protected Routes	Secure APIs using JWT middleware
4	MongoDB Integration	Store and fetch data using Mongoose

17. Roles and Responsibilities

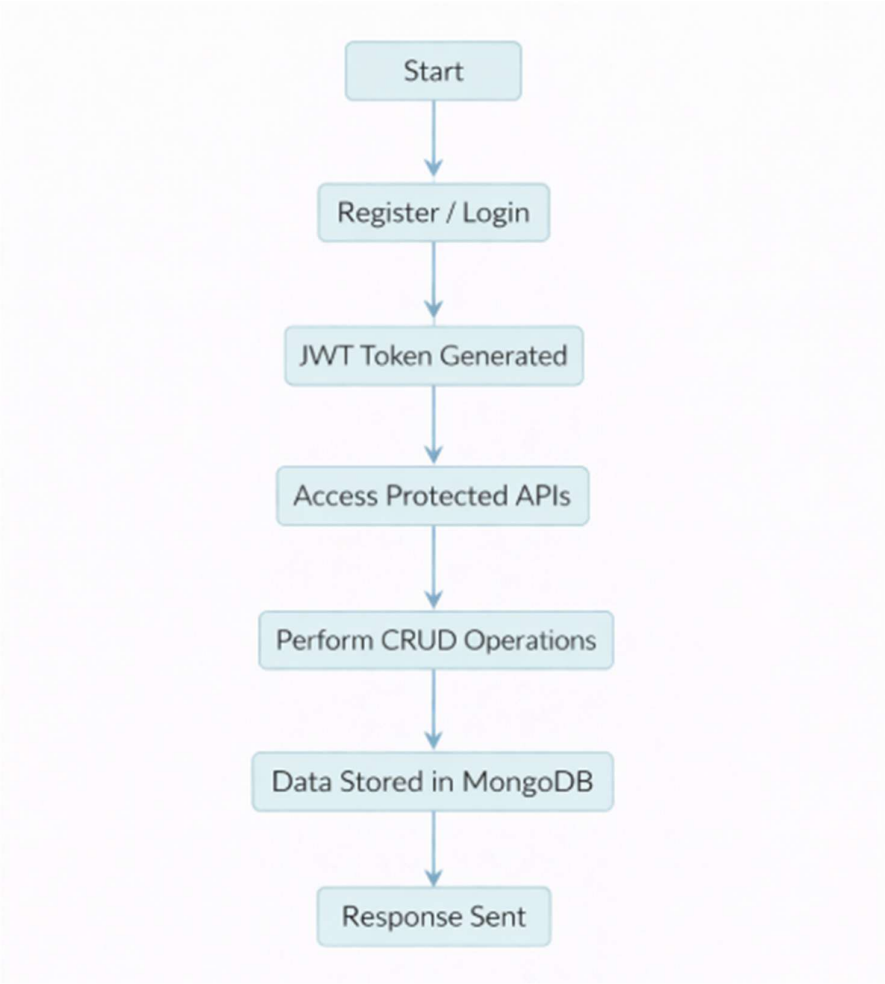
Student / Developer:

- Write backend code
- Create APIs
- Test APIs using Postman/Thunder Client
- Push code to GitHub

System (Backend):

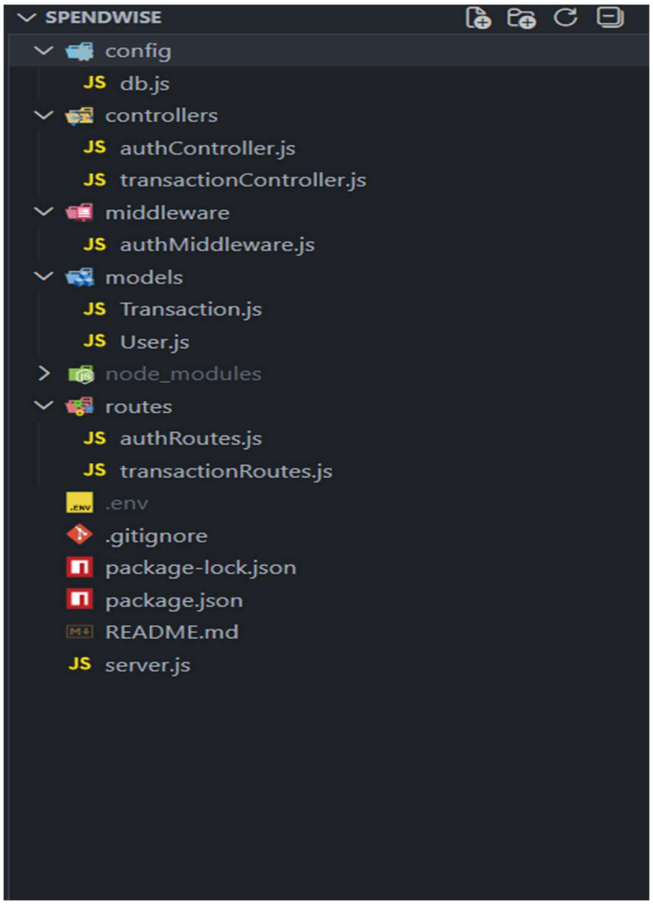
- Authenticate users using JWT
- Handle CRUD operations
- Store and fetch data from MongoDB
- Protect private routes

18. Users Flow:



19. PROJECT SETUP AND CONFIGURATION

Folder Structure



20. Project Setup

1. Install required tools:

- Node.js
- MongoDB

2. Create project folders:

- server folders (models, routes, controllers, middleware, config)

3. Install Backend Packages:

- Express
- Mongoose
- Cors
- Jsonwebtoken
- Bcrypt
- Dotenv

21. BACKEND DEVELOPMENT

- Setup Express server
- Create server.js file
- Create .env file and define PORT and DB URL

```env
PORT=5000
MONGO_URI=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
JWT_SECRET=your_secret_key

```

- Configure middleware (cors, express.json)
- User Authentication:
  - Create Register and Login routes
  - Generate JWT token
  - Protect routes using middleware
- Define API Routes:
  - Auth routes

```
const router = express.Router();

router.post("/register", registerUser);
router.post("/login", loginUser);

module.exports = router;
```

- Transaction CRUD routes

- Implement Data Models:

- User schema

```
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: true,
      minlength: 6,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

○ Transaction schema

```
const transactionSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    amount: {
      type: Number,
      required: true,
    },
    type: {
      type: String,
      enum: ["income", "expense"],
      required: true,
    },
    category: {
      type: String,
      default: "General",
    },
    note: {
      type: String,
    },
    date: {
      type: Date,
      default: Date.now,
    },
  },
  { timestamps: true }
);
```

- Implement CRUD operations
- Add basic error handling

22. DATABASE DEVELOPMENT

Configure MongoDB

- Use MongoDB Atlas or local MongoDB
- Store connection URL in .env file
- Connect using Mongoose

Create Database Connection

- Create a config file for DB connection

```javascript
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

config > JS db.js > ...

```javascript
const mongoose = require("mongoose");

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGO_URI);
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error("MongoDB connection error:", error.message);
        process.exit(1);
    }
};

module.exports = connectDB;
```

- Use mongoose.connect()
- Handle success and error cases

Create Schema and Models

User Model:

- Name
- email
- password

Data / Transaction Model:

- userId
- title / description
- amount / value
- timestamps

23. PROJECT EXECUTION

Project Execution Steps:

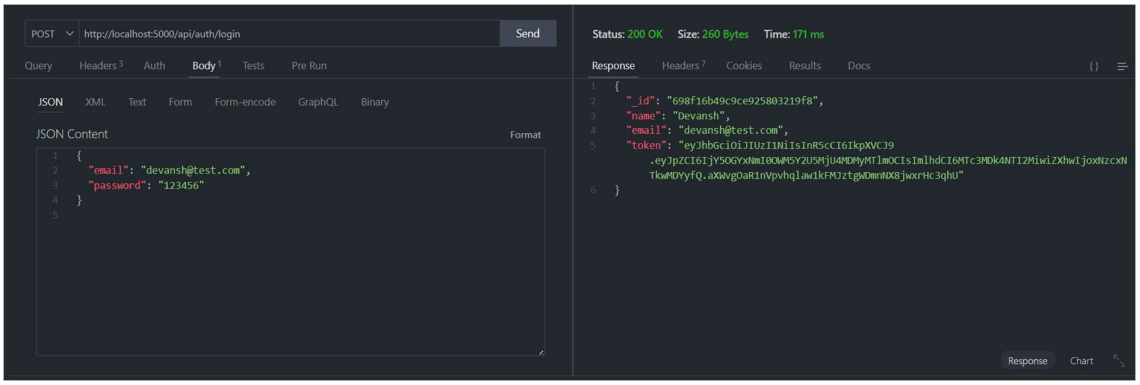
1. Start MongoDB
2. Run backend server

```
PS C:\Users\Devansh\Desktop\spendwise> npm run dev

> spendwise@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ suppress all logs with { quiet: true }
Server running on port 5000
MongoDB Connected: ac-sg1nthw-shard-00-01.s3mna8f.mongodb.net
```

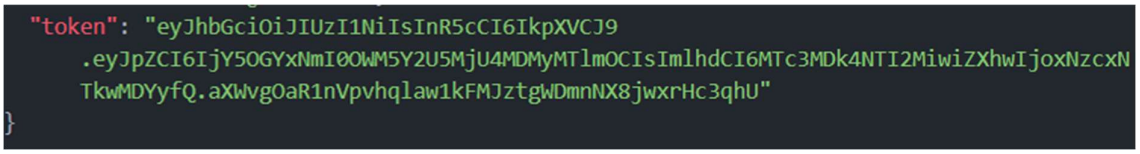
3. Test Register API

```
POST  http://localhost:5000/api/auth/register                                  Send
Query  Headers 3  Auth  Body 1  Tests  Pre Run
JSON  XML  Text  Form  Form-encode  GraphQL  Binary
JSON Content                                                                   Format
{
  "name": "Devansh",
  "email": "devansh@test.com",
  "password": "123456"
}

Status: 201 Created   Size: 260 Bytes   Time: 674 ms
Response  Headers 7  Cookies  Results  Docs                                    {}
{
  "_id": "698f16b49c9ce92580321 9f8",
  "name": "Devansh",
  "email": "devansh@test.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5OGYxNmI0OWM5Y2U5MjU4MDMyMTlmOCIsImlhdCI6MTc3MDk4NTE0MCwiZXhwIjoxNzcxN
      Tg5OTQwfQ.opBpugwO5aabPg86FE-chuzwx8yu39RGbtg9CabDXUs"
}
Response  Chart
```

4. Test Login API



5. Copy JWT token

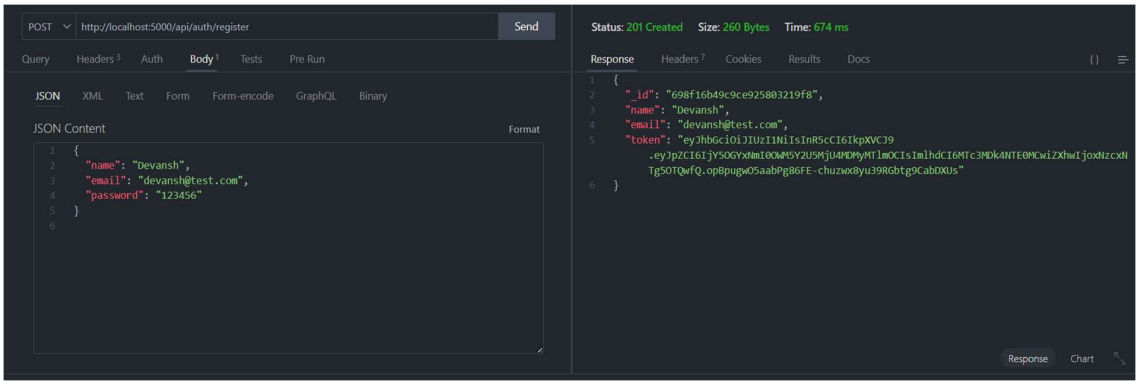


6. Use token to access protected CRUD APIs

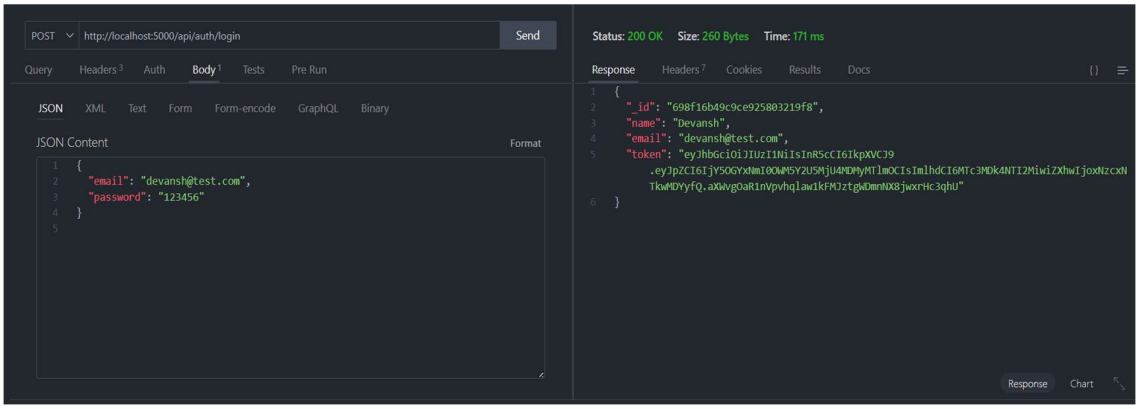
7. Perform Create, Read, Update, Delete operations

24. OUTPUT SCREENSHOT

1. Register API



2. Login API



3. Create Transaction

POST http://localhost:5000/api/transactions Send

Query Headers 4 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "amount": 500,
  "type": "expense",
  "category": "Food",
  "note": "Lunch"
}
```

Status: 201 Created Size: 251 Bytes Time: 178 ms

Response Headers 7 Cookies Results Docs

```
{
  "user": "698f16b49c9ce925803219f8",
  "amount": 500,
  "type": "expense",
  "category": "Food",
  "note": "Lunch",
  "date": "2026-02-13T12:22:38.454Z",
  "_id": "698f178e9c9ce925803219fc",
  "createdAt": "2026-02-13T12:22:38.461Z",
  "updatedAt": "2026-02-13T12:22:38.461Z",
  "__v": 0
}
```

Response Chart

4. Get information

GET http://localhost:5000/api/transactions Send

Query Headers 4 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

Status: 200 OK Size: 253 Bytes Time: 331 ms

Response Headers 7 Cookies Results Docs

```
[
  {
    "_id": "698f178e9c9ce925803219fc",
    "user": "698f16b49c9ce925803219f8",
    "amount": 500,
    "type": "expense",
    "category": "Food",
    "note": "Lunch",
    "date": "2026-02-13T12:22:38.454Z",
    "createdAt": "2026-02-13T12:22:38.461Z",
    "updatedAt": "2026-02-13T12:22:38.461Z",
    "__v": 0
  }
]
```

Response Chart

5. Update API

PUT http://localhost:5000/api/transactions/698f178e9c9ce925803219fc Send

Query Headers 4 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "amount": 600,
  "note": "Lunch + Coffee"
}
```

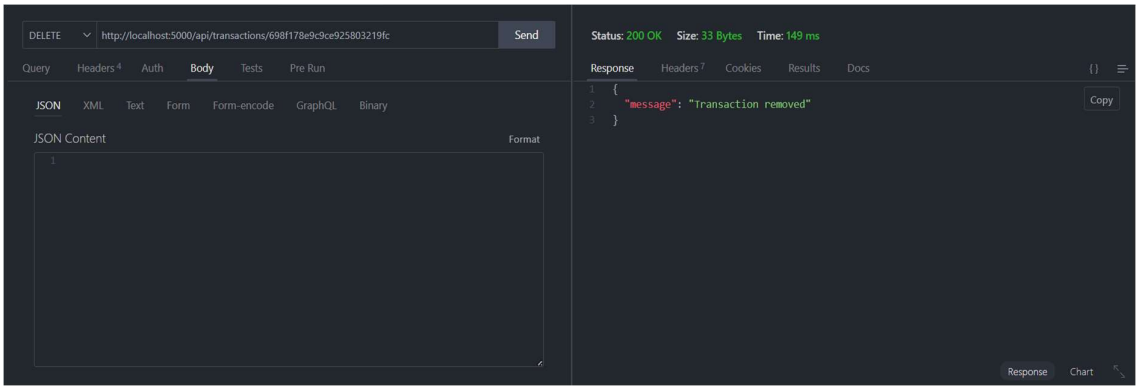
Status: 200 OK Size: 260 Bytes Time: 229 ms

Response Headers 7 Cookies Results Docs

```
{
  "_id": "698f178e9c9ce925803219fc",
  "user": "698f16b49c9ce925803219f8",
  "amount": 600,
  "type": "expense",
  "category": "Food",
  "note": "Lunch + Coffee",
  "date": "2026-02-13T12:22:38.454Z",
  "createdAt": "2026-02-13T12:22:38.461Z",
  "updatedAt": "2026-02-13T12:32:06.265Z",
  "__v": 0
}
```

Response Chart

6. Delete API



25. Project Demo Video

SpendWise (video).mp4

26. Drive Link : SpendWise

o